



Data Engineering

SETUP AIRFLOW



What is Apache Airflow?

 Apache Airflow is the project manager of data pipelines—it makes sure every task shows up on time, follows the plan, and has no excuse for being late. 😊



Apache Airflow

Apache Airflow is an open-source workflow orchestration tool that lets you define, schedule, and monitor complex data pipelines as **DAGs** (Directed Acyclic Graphs) using Python.

Key Features:

- Code your workflows (DAGs) using Python
- Schedule them to run periodically
- Monitor them via a rich web UI
- Supports task dependencies, retries, SLAs, and external triggers

🚀 Setup Apache Airflow on Windows

Why Use Docker for Airflow?

Apache Airflow can be installed using pip, but it has a lot of dependencies like PostgreSQL, Celery, Redis, etc., making the local install **complex and error-prone** — especially on Windows.

Docker simplifies everything:

- ✓ All dependencies are bundled
- ✓ Just 1 command to start your Airflow instance
- ✓ No need to install Python environments, Postgres, etc.
- ✓ Works the same on any OS — Windows, macOS, or Linux

Can I install Airflow directly without Docker?

Yes, you can use [pip install apache-airflow](#), but it's officially recommended only for

Linux-based systems. It often fails on Windows and requires additional setup like WSL or conda-based environments.

If you're on **Windows**, Docker is by far the **simplest and most reliable** way to run Airflow locally.

Prerequisites

Before getting started, make sure you have the following installed on your system:

Docker Desktop → You can Setup/Install Docker desktop using below Link

https://docs.docker.com/desktop/setup/install/windows-install/?source=post_page-----976525faa6ca

Step 1: Create a Dockerfile

This sets up your custom Airflow image with additional packages (like Git):

Dockerfile

```
FROM apache/airflow:2.8.4
USER root
RUN apt-get update && \
    apt-get install -y git && \
    apt-get clean
USER airflow
```

Save this as Dockerfile in your project root.

Step 2: Create docker-compose.yml

This will start Airflow using the standalone mode, which is perfect for local testing

```
version: '3'
services:
  airflow:
    image: my-airflow:latest
    volumes:
      - ./airflow:/opt/airflow
    ports:
      - "8080:8080"
    command: airflow standalone
```

 Any DAGs inside ./airflow/dags will be automatically picked up.

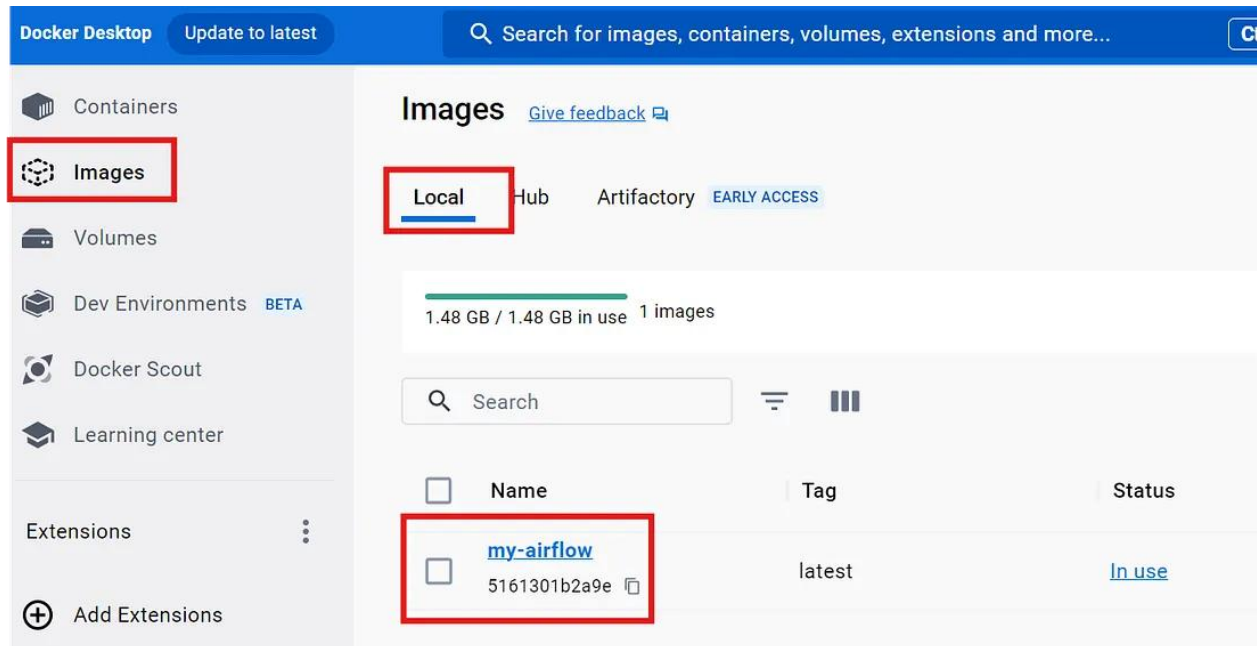
Step 3: Build & Run

Run the following commands from the terminal:

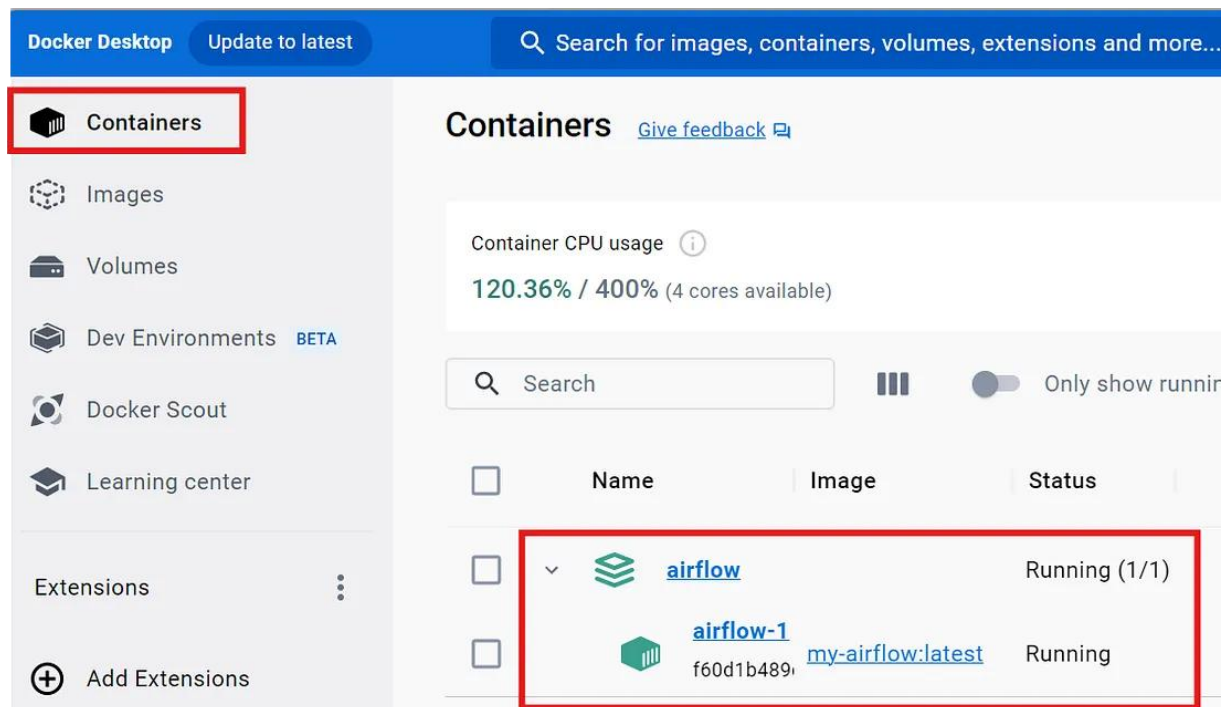
`docker build -t my-airflow .`

`docker-compose up`

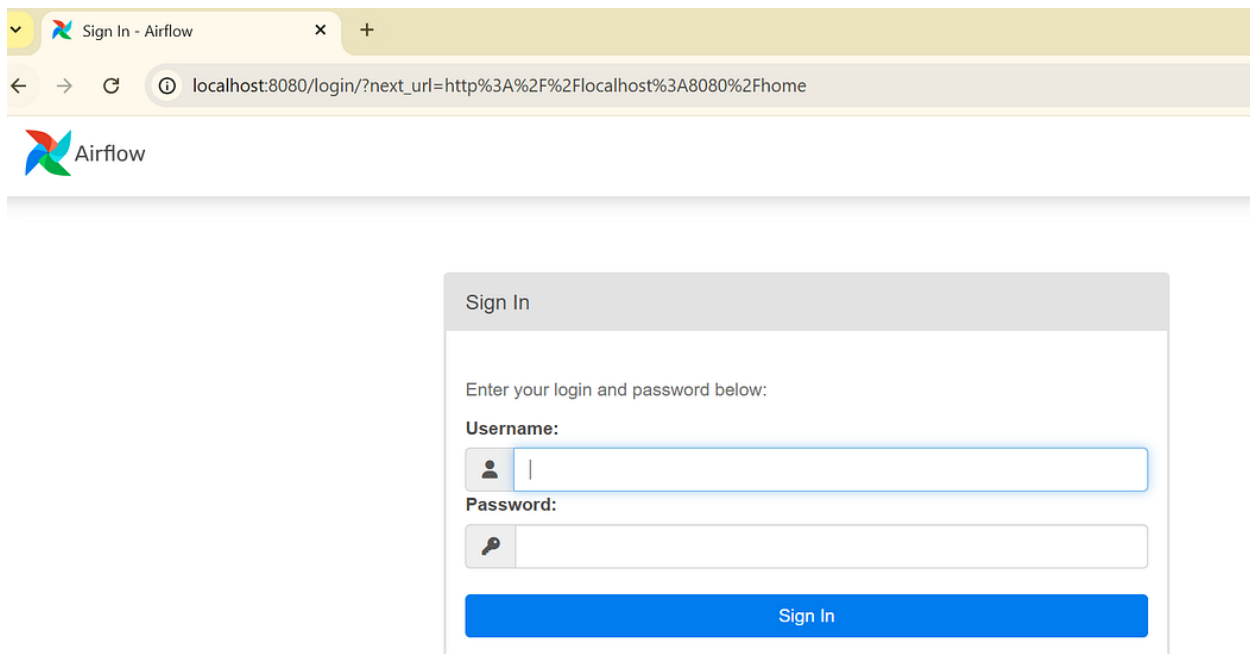
Verify if Image is created locally.



Verify if Container Started



Then open your browser and go to: <http://localhost:8080>.



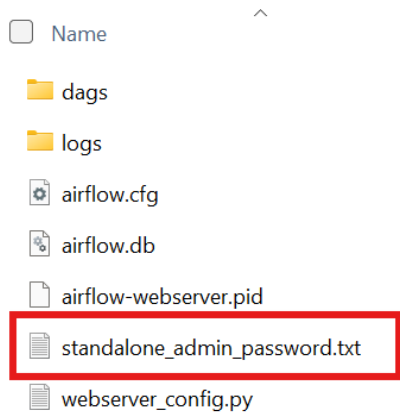
If its not showing UI wait for sometime as it might takes 10-15 minutes as well to start All Airflow components based on your system configuration & performance.

You can check logs & look out for below log message. (If you dont find it in log , dont worry check next steps below.)



If you are not able to find password in log message then use "admin" as username & password will be available in below file

airflow/ standalone_admin_password.txt file



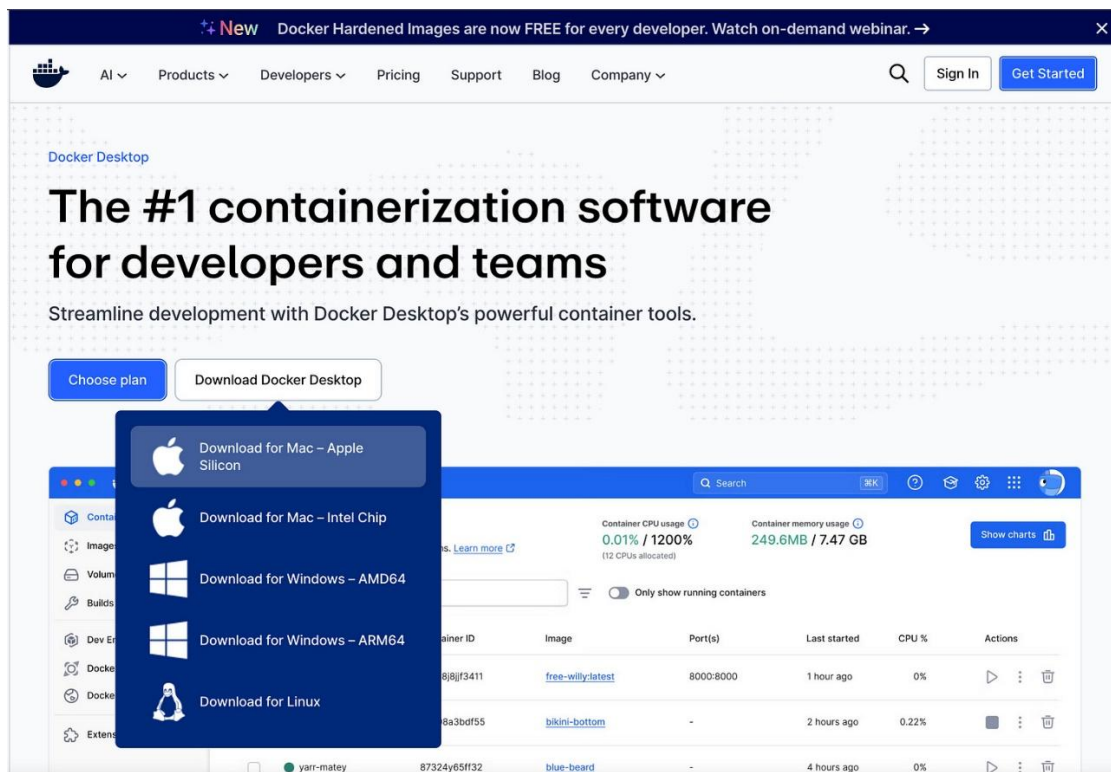
Login and BOOM! You're now ready to use Airflow locally. Go ahead — create, break, and experiment with your DAGs!

Setup Apache Airflow on macOS

Install Docker for Desktop

The **first step** is to install [Docker for Desktop](#).

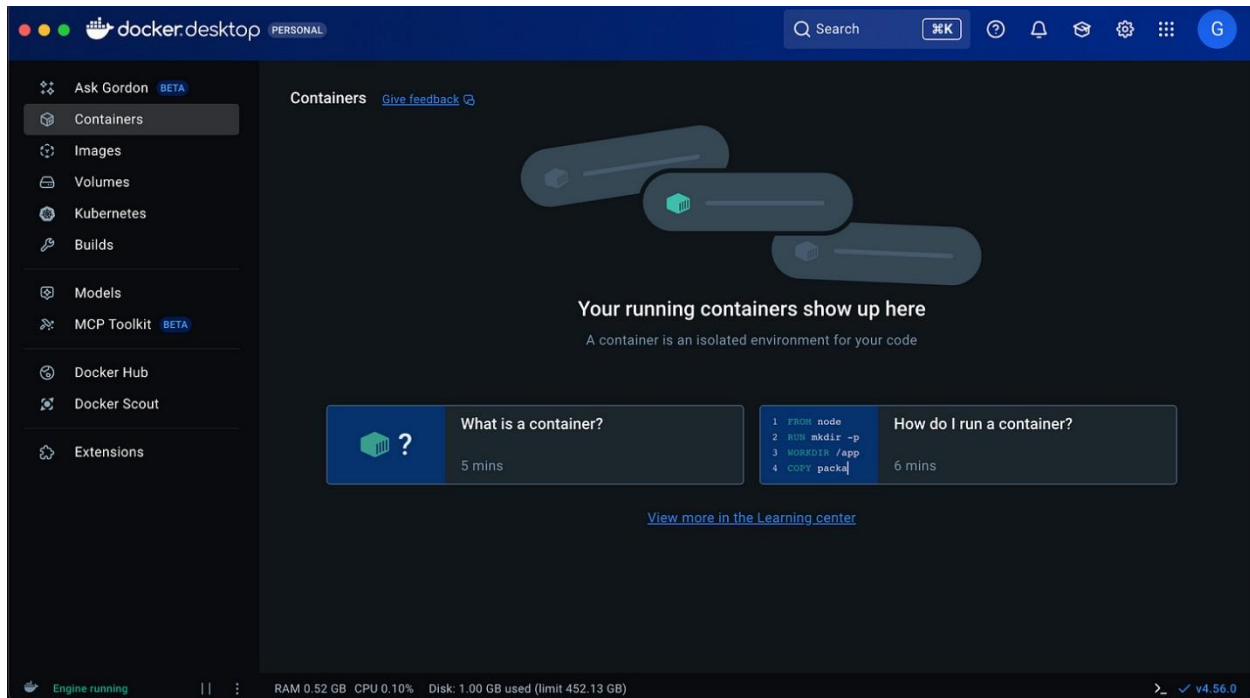
You can download the installer directly from the official website and install it manually,



or — if you prefer using Homebrew — install it using the following command:

```
brew install --cask docker-desktop
```

Once the installation is complete, run the Docker Desktop app, select the recommended settings, subscribe with your personal email (or skip) and you will get to the initial page.



At this point we can open a terminal and verify the installed versions of the Docker components with the following commands.

```
user@MacBook ~ % docker --version
```

```
Docker version 29.1.3, build f52814d
```

```
user@MacBook ~ % docker-compose --version
```

```
Docker Compose version v5.0.0-desktop.1
```

Docker compose

Now we need to provide Docker with a **Compose file** that defines the instructions required to run the various Airflow components.

You can either create your own docker-compose.yaml file manually, or use one provided by the community. To keep things simple and reliable, we'll refer to the **official Apache Airflow documentation**, which includes a [guide to setup Airflow with Docker](#).

The documentation includes a section explaining how [to fetch the docker-compose.yaml](#) file for the Airflow installation, along with a direct link to [download it](#).

The docker-compose.yaml file instructs Docker on how to create a dedicated container for each service in the Airflow architecture, including:

- **airflow-scheduler** — The [scheduler](#) monitors all tasks and Dags, then triggers the task instances once their dependencies are complete.
- **airflow-dag-processor** — Parses DAG files and prepares them for execution.
- **airflow-api-server** — Exposes the Airflow web UI and API at `http://localhost:8080`
- **airflow-worker** — Executes the tasks assigned by the scheduler.
- **airflow-triggerer** — Runs an event loop for deferrable tasks.
- **airflow-init** — Handles initialization tasks.
- **postgres** — The metadata database used by Airflow.
- **redis** — [The redis](#) - broker that forwards messages from scheduler to worker.

Download the file and copy it into a local directory, e.g. `~/Documents/airflow-docker`.

Since we like to know what's going on in our system, let's open the docker-compose.yaml in a text editor and read carefully the key instructions provided with the file.

WARNING: This configuration is for local development. Do not use it in a production deployment.

This is very important.

This setup is not hardened for security, uses default credentials and has no backups, no HA and no secrets management.

It is **perfect for learning and local testing**, but **not production-ready**.

This configuration supports basic configuration using environment variables or an `.env` file

Docker Compose can read variables from your shell environment, or from a `.env` file placed next to the docker-compose.yaml file.

This let you customize the docker-compose behaviour without editing the YAML file itself.

The list of parameters available is well explained in the file itself.

For our environment we will create a new file in the same folder where we created the yaml file (in my case **~/Documents/airflow-docker**) called **.env**

```
user@MacBook ~ % touch .env
```

And we will fill it with the following set of parameters (at the time of writing the default *apache/airflow:3.1.6*, insert the version suggested in the yaml file you downloaded or the version you want to use).

```
AIRFLOW_UID=50000
```

```
AIRFLOW_PROJ_DIR=.
```

```
AIRFLOW_IMAGE_NAME=apache/airflow:3.1.6
```

Now we can open a terminal in the folder where we stored our 2 files and, only for the first run, lunch the following initialization command.

```
docker compose up airflow-init
```

The command will trigger the initialization

```
o airflow-docker % docker compose up airflow-init
[+] up 51/52
✓ Image redis:7.2-bookworm Pulled 22.5s
✓ Image postgres:16 Pulled 41.3s
✓ Image apache/airflow:3.1.6 Pulled 73.3s
✓ Network airflow-docker_default Created 0.0s
✓ Volume airflow-docker_postgres-db-volume Created 0.0s
✓ Container airflow-docker-redis-1 Created 0.2s
✓ Container airflow-docker-postgres-1 Created 0.2s
✓ Container airflow-docker-airflow-init-1 Created 0.1s
Attaching to airflow-init-1
Container airflow-docker-postgres-1 Waiting
Container airflow-docker-redis-1 Waiting
Container airflow-docker-postgres-1 Healthy
Container airflow-docker-redis-1 Healthy
airflow-init-1 |
airflow-init-1 | Creating missing opt dirs if missing:
airflow-init-1 |
airflow-init-1 |
airflow-init-1 | Airflow version:
airflow-init-1 | The container is run as root user. For security, consider using a regular user account.
airflow-init-1 |
airflow-init-1 | /home/airflow/.local/lib/python3.12/site-packages/airflow/providers/common/compat/security/permissions
.py:30 RemovedInAirflow4Warning: The airflow.security.permissions module is deprecated; please see https://airflow.apach
e.org/docs/apache-airflow/stable/security/deprecated_permissions.html
airflow-init-1 | 2026-01-17T21:42:22.177870Z [info ] Context impl PostgresqlImpl. [alembic.runtime.migration] loc
=migration.py:211
airflow-init-1 | 2026-01-17T21:42:22.177978Z [info ] Will assume transactional DDL. [alembic.runtime.migration] loc
=migration.py:214
airflow-init-1 | 2026-01-17T21:42:22.180397Z [info ] Context impl PostgresqlImpl. [alembic.runtime.migration] loc
=migration.py:211
airflow-init-1 | 2026-01-17T21:42:22.180476Z [info ] Will assume transactional DDL. [alembic.runtime.migration] loc
```

Now (and for the next runs) we can start the Airflow environment with the command

```
docker compose up
```

You can now access to your brand new local Airflow environment typing the address `http://localhost:8080/` in your favourite browser.



Sign In

Enter your login and password below:

Username:

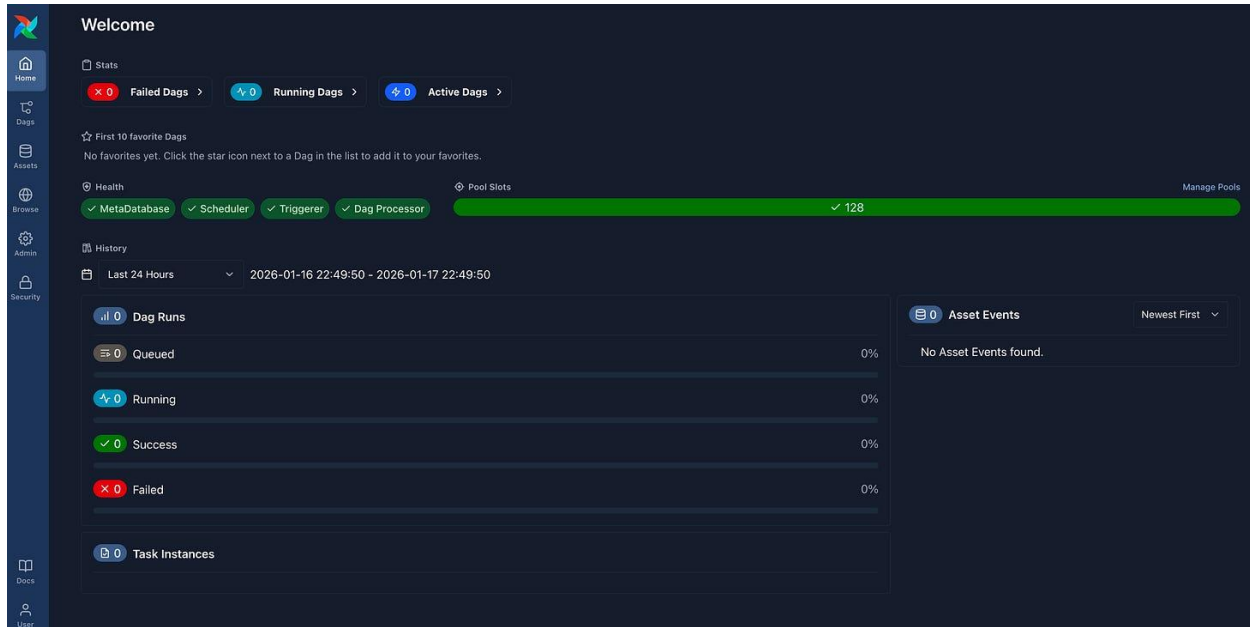
airflow

Password:

.....

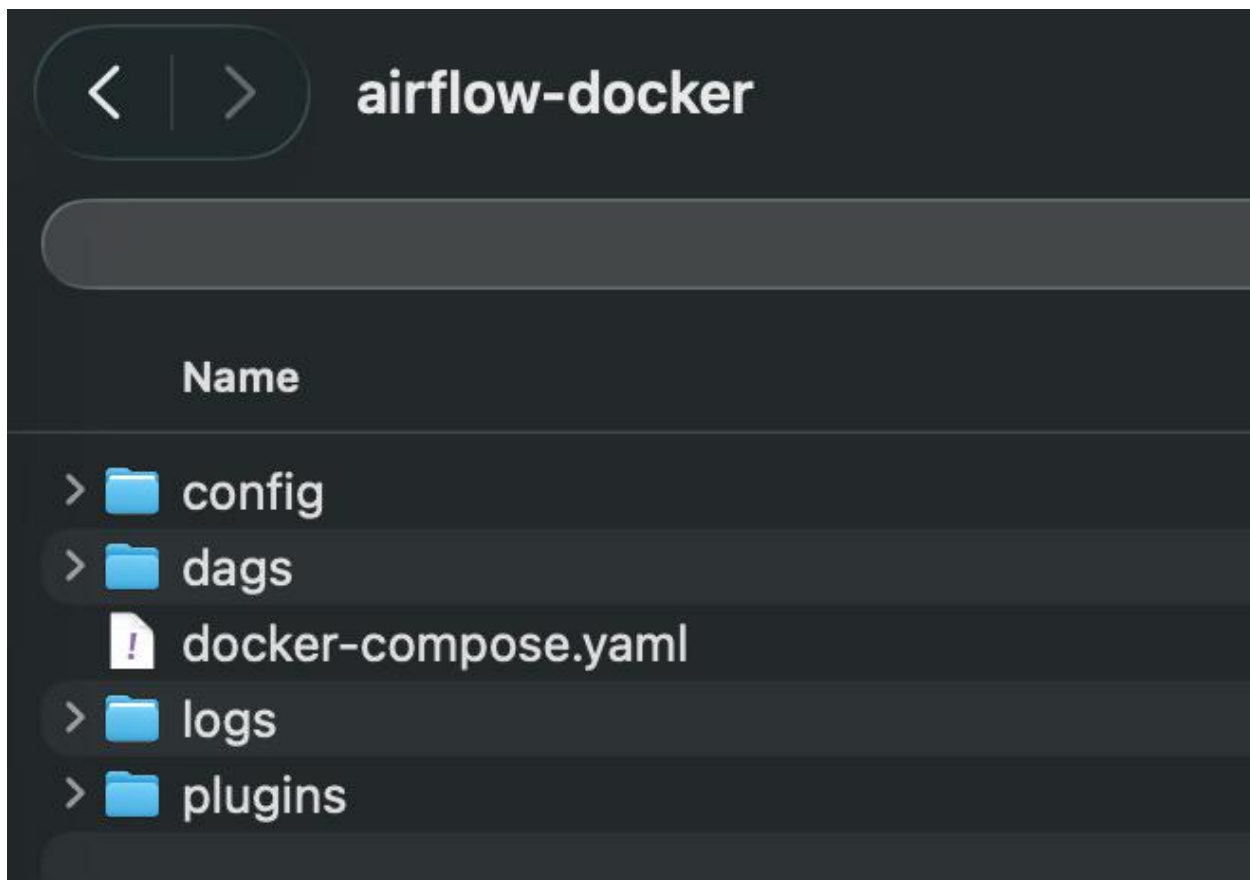
Sign In

The user and password are the default ones that are defined in the docker-compose.yaml file you used (airflow/airflow).

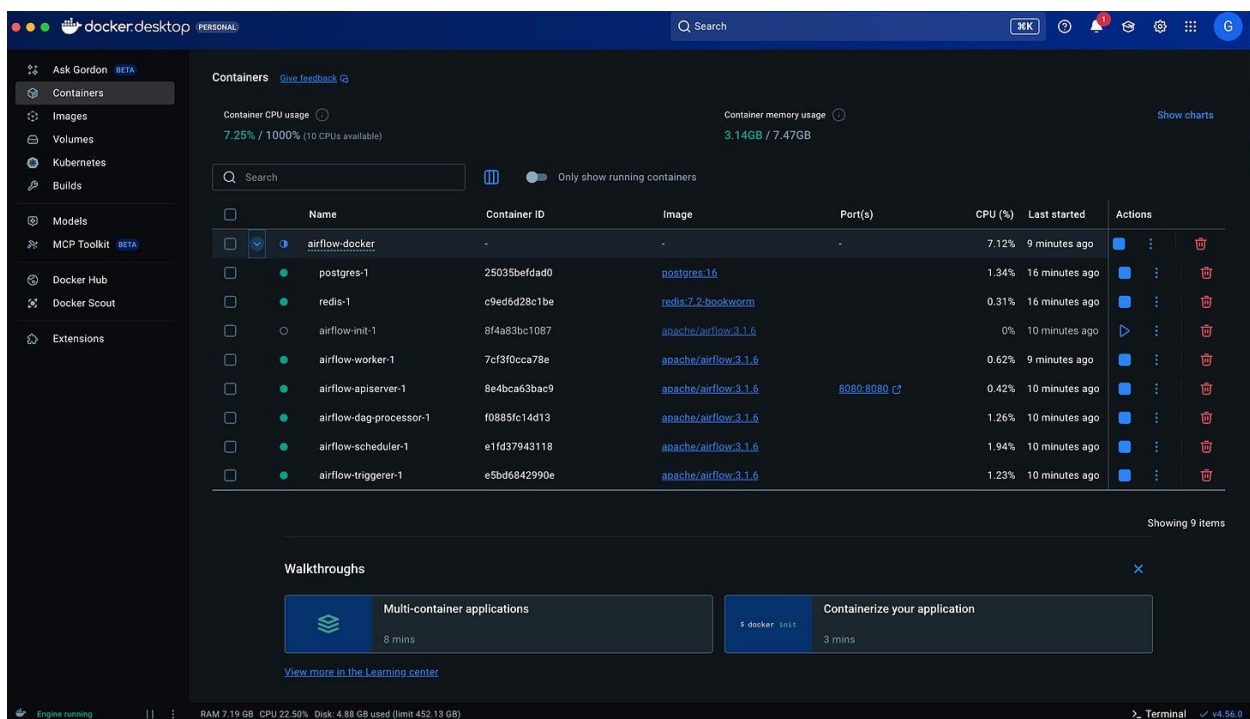


The screenshot shows the Airflow web interface. On the left is a sidebar with navigation links: Home, DAGs, Assets, Browse, Admin, Security, Docs, and User. The main content area is titled 'Welcome' and displays various metrics and status indicators. At the top, there are buttons for 'Failed DAGs' (0), 'Running DAGs' (0), and 'Active DAGs' (0). Below this, a 'First 10 favorite DAGs' section is shown. The 'Health' section indicates that the 'MetaDatabase', 'Scheduler', 'Triggerer', and 'DAG Processor' are all healthy, with a 'Pool Slots' indicator showing 128 slots available. The 'History' section shows a time range from 'Last 24 Hours' to '2026-01-16 22:49:50 - 2026-01-17 22:49:50'. The 'DAG Runs' section shows a list of DAG runs with columns for 'Queued', 'Running', 'Success', and 'Failed', all currently at 0%. The 'Task Instances' section is also visible. On the right, the 'Asset Events' section shows 'No Asset Events found.'

We can notice that starting the environment created a series of new objects in the folder where the yaml file is located that will be used by Airflow to manager our DAGs.



Opening the Docker Desktop application will give us a visual overview on our running Docker containers.



Conclusions

Congratulations! You installed and started your local Airflow environment using Docker.

Setup Apache Airflow on Linux

Installing Apache Airflow on Linux with Docker

Install Docker

- Update your package manager's repository listings.
- Install Docker using your package manager. For example, on Ubuntu, run:

```
sudo apt-get update  
sudo apt-get install docker.io
```

Configure Docker:

- Start the Docker service:

```
sudo systemctl start docker
```

- Configure Docker to start at boot:

```
sudo systemctl enable docker
```

Set Up Airflow:

- Create a directory to store Airflow configurations and DAGs, e.g., `/home/airflow`.
- Pull the Airflow Docker image:

```
docker pull apache/airflow:latest
```

- Launch the Airflow container:

```
docker run -d -p 8080:8080 -v /home/airflow:/opt/airflow apache/airflow:latest
```

- Access the Airflow web interface by visiting `http://localhost:8080` in your web browser.

Configuration and DAGs:

- Customize Airflow's configuration by modifying the `/home/airflow/airflow.cfg` file. Refer to the Airflow documentation for detailed configuration options.

- Manage your DAGs by creating or copying them to the `/home/airflow/dags` directory.

Installing Apache Airflow on Linux without Docker

Set Up Python

- Update your package manager's repository listings.
- Install Python and pip using your package manager. For example, on Ubuntu, run:

```
sudo apt-get update
```

```
sudo apt-get install python3 python3-pip
```

Install Airflow:

- Install Airflow using pip:

```
pip3 install apache-airflow
```

Initialize Airflow:

- Initialize the Airflow database:

```
airflow db init
```

- Create an admin user:

```
airflow users create --username admin --firstname <YOUR_FIRST_NAME> --  
lastname <YOUR_LAST_NAME> --role Admin --email admin@example.com
```

Start the Webserver and Scheduler:

- Start the Airflow webserver:

```
airflow webserver --port 8080
```

- Start the scheduler:

```
airflow scheduler
```

Access Airflow:

- Open your web browser and visit `http://localhost:8080` to access the Airflow web interface.
- Log in using the admin credentials created earlier.

Conclusion

you can successfully install Apache Airflow on Linux, whether you prefer using Docker or installing it natively. Utilize the power of Airflow to streamline your workflows, automate tasks, and enhance your data pipeline management on the Linux platform.